

Жизненный цикл Activity

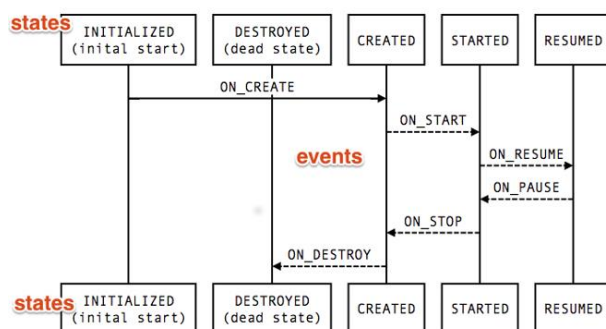
Каждый раз при создании нового приложения по умолчанию создается класс главного экрана приложения — MainActivity.

Когда пользователь нажимает ярлык приложения, система запускает активности, которое указано как стартовое в манифесте приложения. По умолчанию это MainActivity. После запуска мы видим активности на экране. Мы можем свернуть его или открыть другое приложение, экран настроек и т.д. При этом наше активности уходит на второй план и перестает быть видимым. Однако оно не уничтожается, а продолжает работать в фоне, и мы можем в любой момент к нему вернуться. Наконец, мы можем совсем закрыть приложение, или его может закрыть система, если ей будет не хватать оперативной памяти. Тогда активности прекратит свою работу и будет уничтожено.

Кроме того, активности может пересоздаваться — уничтожаться и создаваться заново. Это происходит при событиях изменения конфигурации, таких как поворот устройства из портретной ориентации в альбомную (и обратно), смена размера экрана или языка интерфейса. О смене конфигурации мы более подробно поговорим на следующем уроке.

Схема Lifecycle

А сейчас посмотрите на схему состояний, через которое проходит активности в процессе работы, и событий, которые при этом происходят.



В прямоугольниках указаны состояния (states) активности. А события (events), которые происходят при смене состояний, отмечены стрелками. Важно понимать, что не события управляют состояниями активности, скорее наоборот. Состояния активности изменяются системой Android, а события происходят в процессе изменения.

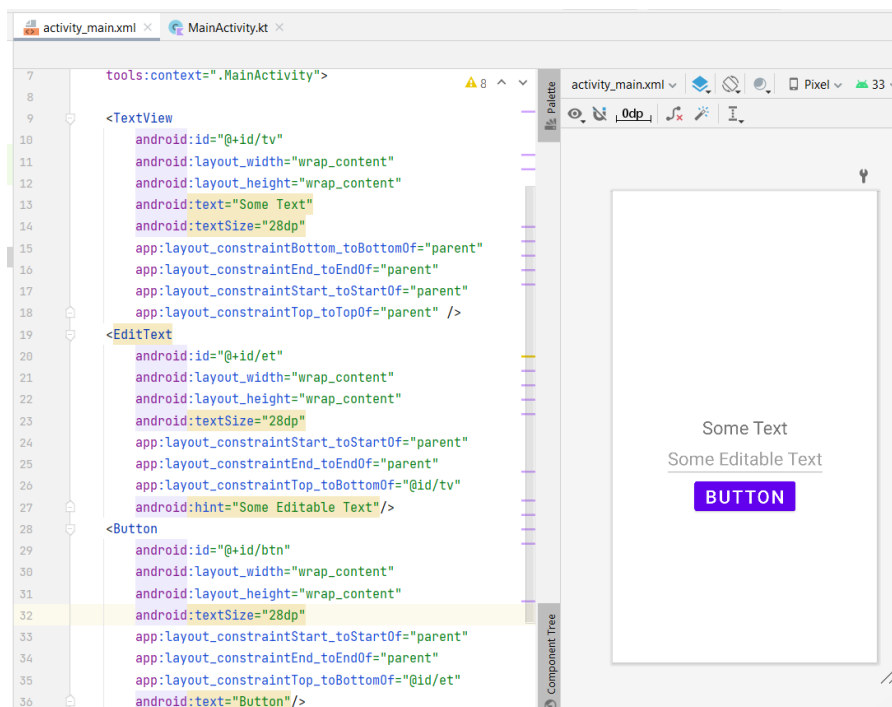
Когда активности запускается системой, оно инициализируется (INITIALIZED) и происходит событие ON_CREATE. При этом активности переходит к состоянию “создано” (CREATED). В этот момент должен инициализироваться пользовательский интерфейс, поскольку активности готовится отобразиться пользователю. Далее происходит событие ON_START и активности переходит к состоянию “запущено” (STARTED). Следующее событие ON_RESUME. Активности переходит в состояние — RESUMED (возобновлено) — выходит на передний план, получает фокус и отображается пользователю. Если активности в процессе работы теряет фокус и частично перекрывается, например, диалоговым окном или другим активити, то переходит обратно в состояние STARTED. При этом происходит событие ON_PAUSE. В этом состоянии активности приостанавливается, но может быть все еще видимым на экране, например, в многооконном режиме. Если же активности

полностью перекрыто, то система его останавливает и переводит в состояние CREATED. Выполняется событие ON_STOP. Активити пока не уничтожается системой и может в любой момент возобновить работу. Но поскольку оно не видно пользователю, то в этом состоянии целесообразно отключать некоторые функции, например, воспроизведение анимации. Если пользователь закрыл активити или система испытывает недостаток памяти, или изменилась конфигурация устройства (произошел поворот), активити уничтожается системой. При этом происходит событие ON_DESTROY. В этот момент необходимо освобождать ресурсы, используемые активити.

Обращение из Kotlin файлов активити к элементам View

Для объявления переменных в kotlin используются свойства такие как val и var. Переменную необходимо объявлять свойством var в том случае, когда переменная в процессе написания программы может менять свое значение. Val используется для неизменяемых переменных

Создадим несколько View в activity_main.xml и присвоим каждому из них id



Теперь наша задача определить и обработать их в MainActivity.kt.

Используем метод findViewById. Переходим в MainActivity.kt и в методе onCreate после привязки xml файла, в котором работаем, прописываем следующее:

```

1 package com.ivakin.lectures
2
3 import ...
4
5 class MainActivity : AppCompatActivity() {
6     override fun onCreate(savedInstanceState: Bundle?) {
7         super.onCreate(savedInstanceState)
8         setContentView(R.layout.activity_main)
9
10        val button = findViewById<Button>(R.id.btn)
11        val editText = findViewById<EditText>(R.id.et)
12        val textView = findViewById<TextView>(R.id.tv)
13    }
14 }

```

Таким образом, мы объявили неизменяемые переменные с названиями button, editText и textView, связанные с id элементов view из activity_main.xml (btn, et, tv). То есть теперь для всех необходимых манипуляций с этими view из kotlin файла, мы будем пользоваться этими переменными.

Получение данных и изменение отображения xml элементов из MainActivity.kt

Для примера сделаем следующее: По нажатию на кнопку button будем брать записанное в editText и дублировать это в textView.

Обработчик нажатия кнопки можно сделать следующим образом:

```

val button = findViewById<Button>(R.id.btn)
val editText = findViewById<EditText>(R.id.et)
val textView = findViewById<TextView>(R.id.tv)

button.setOnClickListener { it: View!
    //do something
}

```

Внутри обработчика заберем текст, написанный в editText, и преобразуем его в тип данных String

```

button.setOnClickListener { it: View!
    val myText = editText.text.toString()
}

```

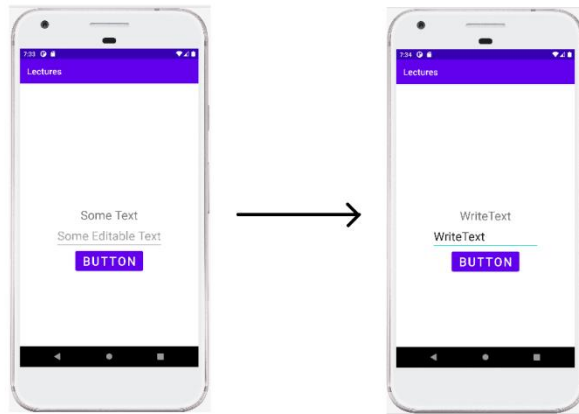
Далее положим эту переменную в textView

```

button.setOnClickListener { it: View!
    val myText = editText.text.toString()
    textView.text = myText
}

```

Запустим наше приложение и убедимся, что поставленная цель достигнута. (Запуск приложения со смартфона или эмулятора будет разбираться на лекции. Также можно писать мне за консультацией, так как в каждом конкретном случае для запуска приложения нужны будут разные действия):



Также можно установить обработчик нажатия и на `textView` и изменить его атрибуты. Например, так будет выглядеть код, где по нажатию на `textView` будет меняться цвет введенного текста:

```
textView.setOnClickListener { it: View!
    textView.setTextColor(resources.getColor(R.color.purple_200))
}
```

Toast Notification (Всплывающее уведомление)

Тост имеет размер, необходимый для отображения сообщения в нем, и отображается поверх активности, которое при этом остается доступным для пользователя. Сообщение отображается несколько секунд и закрывается. Как правило, всплывающее уведомление используется для показа коротких текстовых сообщений. Как правило, всплывающее уведомление используется для показа коротких текстовых сообщений.

Всплывающее уведомление также может быть создано службой, работающей в фоновом режиме, но можно и самим создавать всплывающие сообщения с помощью класса `Toast`.

Инициализируем объект класса `Toast`.

```
Toast toast = Toast.makeText(MainActivity.this, "Some text", Toast.LENGTH_LONG);
```

Статический метод `makeText` создает `View` для отображения текста уведомления.

Параметры метода:

- `MainActivity.this` — context — это объект, который предоставляет доступ к базовым функциям приложения: доступ к ресурсам, к файловой системе, вызов активности и т.д. `Activity` является подклассом `Context`, поэтому в коде мы можем использовать её как `ИмяАктивности.this`, или укороченную запись `this`.

Мы здесь используем `MainActivity.this`, т.е. текущую `Activity`.

- «`Some text`» — text — текст, который надо показать

- `Toast.LENGTH_LONG` — duration – продолжительность показа (`Toast.LENGTH_LONG` — длинная, `Toast.LENGTH_SHORT` — короткая), уведомления выводятся на 3 с половиной секунды или на 2 секунды соответственно. Других вариантов нет, насколько мне известно.

Toast создан и чтобы он отобразился на экране, вызывается метод `show()`.

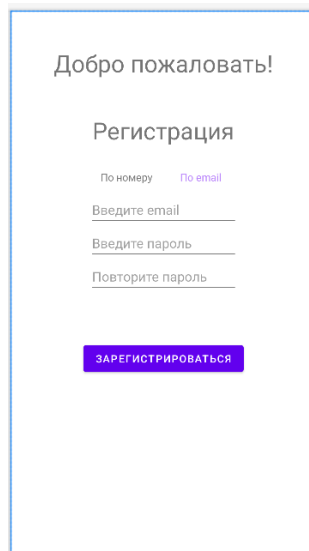
```
toast.show();
```

Перед дублированием текста в `textView` проверим, содержит ли написанное в `editText` слово `Toast` с помощью метода `contains` (остальные необходимые для выполнения лабораторной методы можно найти здесь: <https://kotlinlang.org/api/latest/jvm/stdlib/kotlin/-string/>). Если содержит, отображаем `Toast` с надписью «В тексте есть слово `Toast`»

```
button.setOnClickListener { it: View!
    val myText = editText.text.toString()
    if (myText.contains("Toast")){
        Toast.makeText(context: this, text: "В тексте есть слово Toast", Toast.LENGTH_LONG).show()
    }
    textView.text = myText
}
```

Лабораторная работа №2

Обработать View на экране регистрации, созданном в предыдущей лабораторной.



1. По нажатию на кнопки «По номеру» и «По email» подсвечивать цветом выбранный режим, менять hint у editText в зависимости от того, что выбрано, также менять свойство inputType (см. предыдущую лекцию)
2. По нажатию на кнопку «Зарегистрироваться» проверять поля ввода на корректность, в случае неверного ввода показывать Toast с соответствующим сообщением. Минимальные требования корректности ввода:

- Проверка email на содержание символа @

- Проверка телефона на содержание символа +

- Проверка пароля на количество символов (минимум 8)

- Проверка, равны ли строки, взятые из пароля и его повторения

В случае корректно введенных данных ничего не происходит, обработка корректного ввода будет в следующей лабораторной

По всем дополнительным вопросам можно писать мне в Telegram.